Philippe Gonzalez, phigon@dtu.dk
Valeska Slomianka, vaslo@dtu.dk
Quirin Mühlberger, qumu@dtu.dk

# EXERCISE 1: DIGITAL SIGNAL PROCESSING

*Hearing Systems, Technical University of Denmark*

February 9th, 2026

## 1. INTRODUCTION

### 1.1. Exercise overview

In this exercise you will learn how to use MATLAB for various digital signal processing tasks. It is assumed that you are already familiar with MATLAB and the material presented in the `Matlab_introduction` folder on DTU Learn before you start this exercise. If you are not, start with the introduction to MATLAB and return to this exercise afterwards.

Section 2 of this exercise deals with the discretization of analog signals. In this section, you will specifically investigate quantization noise, the Nyquist sampling theorem and the effects of aliasing. In Section 3, properties of the discrete Fourier transform (DFT) will be examined. Two days are scheduled in total for this exercise, so we do not expect, nor do we want you to finish the whole exercise in just one session. If you follow the exercises in the listed order, a good objective for you is to have completed Section 2 by the end of the first session. In general though, take your time and discuss things in your group so that you really gain an understanding of the concepts.

### 1.2. Exercise objectives

After completion of this lab exercise it is expected that you should be able to:

- Understand quantization.
- Explain aliasing and the Nyquist sampling theorem.
- Apply anti-aliasing and reconstruction filters.
- Understand the real and imaginary part of the Fourier transform.
- Visualize the Fourier transform of periodic signals.
- Describe the effects of windowing.
- Understand the short-time Fourier transform.

### 1.3. Exercise setup

1. Connect a pair of headphones or earphones to your computer, preferably using a cable.
2. Ensure that the sound output of your computer is configured to your headphones and that the volume is adjusted to a comfortable level, but not exceeding 80%.

**WARNING**

Always check the volume levels on your computer **before** putting the headphones on in order to ensure that the output level is at a safe level for your ears!

1. Download the exercise material from DTU Learn.
2. Start MATLAB.
3. Change the current directory to the path of the `DSP` folder in the material you downloaded.
4. Enter the command `startup` into the command window of MATLAB.

## 2. PART 1: DISCRETIZATION

The conversion of a continuous (analog) signal to a discrete (digital) signal (analog-to-digital, or A/D conversion) means to discretize both the signal amplitude (i.e., `quantization`) and the signal's evolution over time (i.e., `sampling`). Both discretization steps have to fulfill certain criteria and can only be done to a certain precision. In the following you will investigate these criteria and observe what happens when they are violated.

### 2.1. Quantization noise

This section deals with the quantization of an analog signal which means to encode the signal amplitude with a set of discrete values. An analog-to-digital (A/D) converter turns an analog signal into a sequence of binary numbers, where each binary number represents one level from a set of specific voltage levels. Because the analog signal is rounded to these specific levels, the quantized signal inherently contains rounding errors, which are called quantization noise.

In order to provide the best signal representation, quantization noise should be kept as low as possible. The ratio of the signal amplitude to the amplitude of the quantization noise (signal-to-noise ratio, SNR) is a quality measure

of the quantization. The SNR is determined by the number of bits used to code the analog signal. For example, signals sampled with a 16-bit converter will be discretized into $2^{16}$ different voltage levels, and thus have an SNR of about 96 dB (= $20 \log_{10}[2^{16}/2^0]$).Quantization noise is also introduced when a digital signal is discretized a second time with a lower bit resolution.

There are different methods to implement quantization - we will use a method referred to as mid-rise quantization. This type of quantization does not encode the value zero, but instead allows us to use all the available bits within the signal range. Mid-rise quantization can be implemented using the following procedure:

1. Define the step size between each quantization level as $\Delta = \frac{R}{2^b - 1}$, where $R$ is the amplitude range of the signal to be encoded.
2. Divide the vector containing the sampled signal by $\Delta$.
3. Round each element in the new vector to their nearest integers using the MATLAB `floor` command, and add $1/2$ to the result.
4. Multiply the rounded vector by $\Delta$.

#### 2.1.1. *Questions*

- **[IMPLEMENT]** Implement a function called `quantization` that changes the bit resolution of an input signal given the dynamic range, $R$, and the number of bits, $b$. Please write this function from scratch without using the MATLAB function `quantize`.
- **[RESULTS]** Is there an optimal way to select $R$ given some input signal? Is it in practice always possible to use this optimal $R$?
- **[RESULTS]** With this `quantization` function, quantize a signal with different bit rates. Compare the resulting signals and quantization noise. Listen to the signals using the `sound` command. Listen as well to the quantization noise alone.
- **[RESULTS]** Apply a digital gain to the signal by multiplying it with a constant. Use the `quantization` function to convert the signals to a lower bit rate (e.g. three bits per sample) while keeping the range $R$ constant - i.e. you should decide on $R$ *before* applying the gain. How does the digital gain affect the resulting signals and quantization noise? Try both gains that are less than and greater than one.
- **[IMPLEMENT]** So far, we have not considered the fact that we only have a limited number of bits available. For the mid-rise quantifier, the maximum level that can be coded is $L = R/2$ and the minimum level is $-L$ (Why is this?). Implement this limit by setting all samples that are above $L$ to $L$, and all samples below $-L$ to $-L$. How does this limit affect the observations about digital gain that you made before?
- **[RESULTS]** Import a speech sentence and quantize it with a bit rate of one. Listen to the quantized signal. Then create a version with two bits per sample, and listen to it. Repeat the process, each time with a higher bit rate, until you can understand all of the words in the sentence.
- **[DISCUSS]** Explain what your `quantization` function does, and how the operation in this function relates to quantization in A/D conversion.
- **[DISCUSS]** How does the bit rate affect the signal quality?
- **[DISCUSS]** How does the amplitude of the signal affect the SNR?
- **[DISCUSS]** The process of digitally recording sound is, by definition, A/D conversion. How would you optimize the level of music or speech in order to obtain the highest quality recorded signal?
- **[DISCUSS]** What is the minimal bit resolution you need to understand speech?

### 2.2. Nyquist sampling theorem and aliasing

The operation of sampling an analog signal $x(t)$ in the time domain is equal to multiplying the signal with a pulse train of period $T$. This results in a discrete-time representation $x(nT)$ of the signal, sampled at a rate of $f_s = \frac{1}{T}$. As a consequence, the spectrum $X(f)$ of $x(nT)$ contains a copy of the original spectrum of $x(t)$ spaced $f_s$ apart.

When the replications overlap, different signals become indistinguishable (or aliases of one another) once they have been sampled. This effect is called "aliasing". The word "aliasing" is also sometimes used to describe the perceptible distortions or artifacts that are introduced in signals that have been reconstructed from samples with overlapping spectral replications.

The Nyquist sampling theorem defines a set of criteria that must be satisfied in order to avoid overlapping spectral replications (i.e., to avoid aliasing). The theorem states that for an accurate discrete-time representation of an analog signal, two conditions must be met:

i. The signal $x(t)$ must be bandlimited, thus containing only frequencies up to a maximum frequency $f_{max}$.

ii. The sampling rate $f_s$ must fulfill the condition $f_s \geq 2f_{max}$.

#### 2.2.1. *Questions*

- **[RESULTS]** Enter `smpldemo` in the MATLAB command line. This command will initiate a figure window containing a demonstration that illustrates well the effects of sampling (i.e., spectral periodicity and aliasing) in both the time and frequency domain.

- **[DISCUSS]** Explain sampling as illustrated by the series of figures in the demonstration slides.

### 2.3. Anti-aliasing and reconstruction filters

The previous demonstration illustrated the effects of aliasing (i.e., how frequencies above the sampling criterion are mapped onto lower frequency bins and the impact of spectral replicas on the signal reconstruction). In common digital signal processing (DSP) systems, anti-aliasing (lowpass) filters are applied before A/D conversion to avoid potential aliasing effects. Additionally, reconstruction (lowpass) filters are applied after D/A conversion to suppress high frequency components that are introduced by the reconstruction process (i.e. zero-order hold).

Since the wav-files are already digital signals, sampling effects can be simulated by downsampling them to a new (lower) sampling frequency $f_{s_{new}}$. This is analogous to sampling an analog signal during A/D conversion. The proper way to downsample a digital signal is to throw away data outside the new sampling (or Nyquist) interval, then play back the signal at the new sampling frequency. However, most modern sound cards have built-in reconstruction filters that cannot be disabled. These built-in filters would by default remove all frequency components (spectral replica) outside of the new Nyquist interval. In order to avoid this in the current exercise, you will "trick" the sound card into playing a signal sampled at $f_{s_{new}}$ without the effects of the filtering by replacing samples with zeros (rather than removing the samples altogether) and playing at the original sampling frequency $f_s$. Thus the sound card reconstruction filters work on the original frequency range associated with the original signal, and it will be possible then to observe the effect of aliasing due to the overlap of the spectral replications. To do so, the following procedure is used:

1. Read a wav-file and figure out its sampling rate $f_s$.

2. Generate a new vector full of zeros, having the same size as the vector with the sound sample (use `zeros( size(X))`).

3. To simulate a sampling rate reduced by a factor of $N$ (i.e., $f_{s_{new}} = f_s/N$), copy each $N^{\text{th}}$ sample of the sound vector to the corresponding position in the new vector. To do so, use a command like `sig_new(1:N:end) = sig_orig(1:N:end)`.

4. Remember to use the original sampling rate $f_s$ when you listen to the downsampled signal (`sound( sig_new,fs)`).

#### 2.3.1. *Questions*

- **[IMPLEMENT]** Write a small function called `dwnsmpl`, which follows the procedure described above to lower the sampling rate. Use this function to listen to the file `'Sweep.wav'` at lower sampling rates. This is a sweep in frequency from 0.1 to 8 kHz and is the most appropriate signal to clearly observe aliasing and spectral replication. The other wav-files are all highly non-stationary and random in nature, which makes it harder to observe these effects.

- **[RESULTS]** Once you understand the effects of the downsampling operation for the sweep signal, listen to other wav-files at lower sampling rates. When you listen to the modified signal you should also be able to hear the described artifacts — presuming your new sampling rate is sufficiently low.

- **[IMPLEMENT]** Next, create a script that samples in accordance with the Nyquist sampling theorem. Please do not use the MATLAB function `downsample`. To create this script, you will need to use the lowpass filter function `lowpassfilt` (type `help lowpassfilt` in the MATLAB command window to open the help document for the function) to create an anti-aliasing filter. Use a filter order of 16. Apply this anti-aliasing filter to the signal prior to evaluating the `dwnsmpl` function you wrote. Compare the results of downsampling with and without the anti-aliasing filter by listening to the signals.

- **[RESULTS]** Enter `recodemo` in the MATLAB command line and read through the demonstration slides.

- **[IMPLEMENT]** Now add a signal reconstruction stage to your script by including an additional reconstruction filter after the dwnsmpl operation, using `lowpassfilt`. Again, use a filter order of 16. Compare the results of downsampling with and without the reconstruction filter by listening to the signals.

- **[RESULTS]** Finally, use the script to apply both an anti-aliasing filter and a reconstruction filter. Compare the results of downsampling with both of these filters to downsampling with only one or none of them.

- **[RESULTS]** Using the function `sgram( sig,fs,dynrange)`, compare the spectrograms of the sweep signal before and after downsampling, with and without anti-aliasing and reconstruction filters. Note, it is important to limit the dynamic range (i.e., adjust

`dynrange`) of the spectrogram in order to properly visualize the signals. A dynamic range of 39dB is reasonable here.

- **[DISCUSS]** How do the spectrograms illustrate the necessity of the anti-aliasing and reconstruction filters?
- **[DISCUSS]** In any digital signal processing system, an anti-aliasing filter is used before A/D conversion and a reconstruction filter is used after the D/A conversion. Why?
- **[DISCUSS]** How does an A/D–D/A converted signal sound when it is not sampled in accordance with the Nyquist Sampling Theorem? Is this changed by the reconstruction filter?
- **[DISCUSS]** If an A/D–D/A converted signal has been sampled in accordance with the Nyquist sampling theorem, do we still need to use a reconstruction filter?
- **[DISCUSS]** What are the requirements on the sampling frequency in order to avoid hearing aliasing effects when recording music? Does this change when recording speech?

## 3. PART 2: DISCRETE FOURIER TRANSFORM

In this part of the exercise you will explore some properties of the discrete Fourier transform (DFT). Examples of different effects of windowing in the time and frequency domain will also be investigated.

### 3.1. Real and Imaginary Part of the Fourier Transform

#### *3.1.1. Questions*

- **[RESULTS]** To start the demonstration, type `DFTdemo1` inside the MATLAB command window. The first slide shows a sinusoid, where the frequency is an integer multiple of the inverse duration of the whole signal. Below its time series, the real and the imaginary part of the spectrum are shown. You can compare this sinusoid with a cosine function (middle panel) and a sinusoid with a frequency that is four times higher (right panel). The next slide shows the time series for different rectangular functions together with the real and imaginary parts of their spectra. The first two functions are centered with respect to the time-zero, but the durations of the rectangular pulses differ. The last one is a time-shifted rectangular pulse. Delta pulses and pulse trains are the subject of the next slide. Following this, a slide with a Gaussian pulse, a Gaussian-modulated sine and a modulated Gaussian pulse train are shown.
- **[RESULTS]** Practice generating, transforming, and plotting periodic signals:
  1. Build a time vector from 0 ms to 10 ms with a sampling frequency of 10 kHz.
  2. Use this time vector to generate a sinusoid with a frequency of 1 kHz.
  3. Plot the sinusoid as a function of time.
  4. Plot each second value of the sinusoid and add an additional graph with each fourth value of the sinusoid.
  5. Compute a time signal (1 s, 10 kHz) consisting of two sinusoids (250 Hz and 260 Hz) and plot it as a function of time in ms.
  6. Plot two cycles of a sine wave that has a frequency of 100 Hz, peak amplitude of 1, and a starting phase of 180°(10 kHz sampling frequency).
  7. Add a cosine wave to the plot with the same peak amplitude and starting phase, but twice the frequency.
  8. Plot the real and the complex part of the spectrum of both signals. Note that you have to generate the frequency axis on your own. Hint: Keep in mind that the discrete Fourier transform generates a number of frequency bins $N$ equal to the DFT's length. To center the result of an `fft` operation, use the MATLAB function `fftshift`. Second hint: The online documentation for `fftshift` shows an example of how to generate the frequency axis.
  9. Now add the two signals in the time domain, and plot the resulting signal and its power spectrum [1]. Note that only the positive frequencies are of interest.
- **[DISCUSS]** Explain the effects that can be seen in the spectra in the second slide of the demo. Which properties of the Fourier transform are illustrated?
- **[DISCUSS]** What are the differences between the delta pulses and pulse trains?
- **[DISCUSS]** How do these differences between delta pulses and pulse trains affect the real and the imaginary parts of the spectra?
- **[DISCUSS]** Can you define the Gaussian-modulated sine wave time series mathematically just by using the Gaussian pulse, the pulse train and a sinusoid?
- **[DISCUSS]** How do the mathematical expressions look when transferred to the spectral domain? Note that it is sufficient to use symbolic representations (e.g. ⊔⊔ for a pulse train).

[1] Here actually the *energy density spectrum* is considered rather than the *power spectrum* (= $|\mathcal{F}\{x\}|^2$). However, the term *power spectrum* is most commonly used throughout English literature.

### 3.2. Fourier Transforms of Periodic Signals

*3.2.1. Questions*

- **[RESULTS]** Type `DFTdemo2` inside the MATLAB command window to start the demo.
- **[DISCUSS]** Slide 1 of the demo shows a sine and a cosine function, each having an integer number of cycles in the analysis window. Is it possible to distinguish between the sine and the cosine in Slide 1 just by looking at the power spectrum?
- **[DISCUSS]** What happens if the analysis window is shorter (as shown in Slide 2), such that an integer number of cycles does not fit?
- **[DISCUSS]** Compare the spectra on Slide 2. Why is it now possible to distinguish between the power spectrum of the two sine time series?
- **[DISCUSS]** The left panels in Slide 3 show the time series and the power spectrum of a combination of two sinusoids. The formula used to calculate the signal is shown above the time course. What frequencies would you expect just by looking at the formula?
- **[DISCUSS]** Zoom in to check whether the frequency maxima in the power spectrum fit the expected frequencies. Why are they different?

### 3.3. Fourier Transforms of Stochastic Signals

*3.3.1. Questions*

- **[RESULTS]** Practice generating, transforming, and plotting noise signals:
  1. Plot a noise signal $x(n)$ (1000 samples, variance 1).
  2. Plot the power spectrum of $x(n)$.
  3. Plot the same power spectrum in dB with logarithmic abscissa.
  4. Do the same for a noise signal $y(n)$ (10000 samples, variance 1).
- **[DISCUSS]** The right-hand side of Slide 4 in `DFTdemo2` shows white noise and its power spectrum. Is the power spectrum of the white noise just a flat line, like the power spectrum of a delta pulse? Why not?
- **[DISCUSS]** Would it become more flat using a longer duration? Hint: Remember that white noise is a stochastic signal (i.e., a single observation has random amplitude values).
- What can we do to get a more flat presentation of the power spectrum of noise?

### 3.4. Windowing and the Short-Time Fourier Transform

*3.4.1. Questions*

- **[RESULTS]** Type `DFTdemo3` inside the MATLAB command window to start the demo. Slides 1 to 9 demonstrate the effects of different window functions.
- **[RESULTS]** Type `DFTdemo4` inside the MATLAB command window to start the demo. Slides 1 to 6 of the demonstration show Fourier transforms and short-time Fourier transforms of sweep signals.
- **[RESULTS]** Practice generating, transforming, and plotting sweep signals:
  1. Plot a linear rising sweep from 200 Hz to 1000 Hz with a duration of 100 ms at a sample rate of 10 kHz. Please do not use MATLAB's internal functions like `chirp`. Hint: The key idea of a linear sweep is that the fundamental frequency of the sinusoid changes constantly over time. Mathematically, this corresponds to $\partial\varphi(t)/\partial t = 2\pi f_{\text{start}} + \frac{t}{L} \cdot (2\pi f_{\text{end}} - 2\pi f_{\text{start}})$, with $L$ being the duration of the sweep and $\varphi(t)$ the phase of the sinusoid (i.e. $sweep(t) = \sin[\varphi(t)]$).
  2. Plot its power spectrum with the corresponding frequency axis.
  3. Now use windowing to plot at least five successive short-term power spectra of the sweep signal. Use different window functions.
- **[RESULTS]** Practice transforming and plotting speech signals:
  1. Plot one of the speech samples as a function of time and its power spectrum.
  2. Use windowing to plot successive short-term power spectra of the speech sample. Try different window lengths.
  3. Finally, use the same `sgram` function as in Section 2.3 to plot a spectrogram of the speech sample. Read the help for more information about `sgram`.
- **[DISCUSS]** Slide 1 of `DFTdemo3` shows a rectangular window and a sinc window in the time domain with their corresponding power spectra. Why does the sinc window not result in a perfect rectangular function in the power spectrum?
- **[DISCUSS]** Slides 2 to 9 show Hamming and Hann windows with and without zero padding, as well as what happens when these windows are applied to a sinusoid. What are the advantages and disadvantages of the corresponding windows?

- **[DISCUSS]** Can you explain what happens in the spectral domain when you add zeros to a window in the time domain?
- **[DISCUSS]** Is the choice of the window important for the shape of the power spectrum?
- **[DISCUSS]** Is it in principle possible to distinguish between an up- and down-sweep just by looking at the power spectrum in `DFTdemo4`? If not, why?
- **[DISCUSS]** What are the differences between the short-time Fourier transforms at different times of the sweep signal?
- **[DISCUSS]** Slide 6 of `DFTdemo4` shows the spectrograms of the same sweep signals together with an example spectrogram for a short speech signal. How do the spectrogram representations of the sweeps relate to the short-time Fourier transform of the previous slides? Which kind of information do they provide that the Fourier transform does not?